

PRACTICE EXERCISES OF THE MICROPROCESSORS & MICROCONTROLLERS

Instructor: The Tung Than

Student's name: Văn Nhật Tân

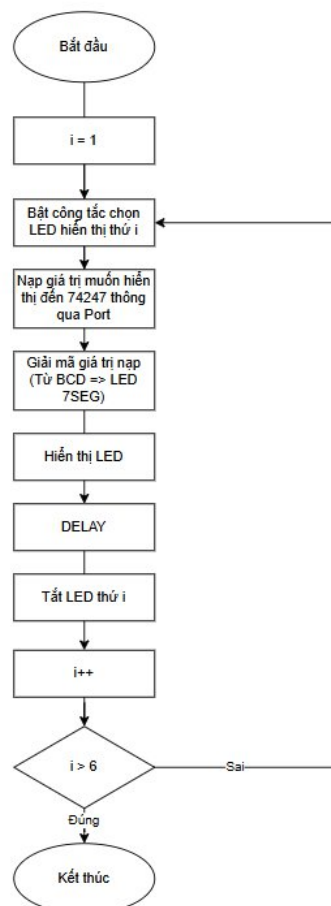
Student code: 24521587

PRACTICE EXERCISE #2:

COMMUNICATION WITH 7-SEGMENT LED AND TIMER

I. Present and draw a flowchart of the LED scanning algorithm applied to display 7-segment led. (2 points)

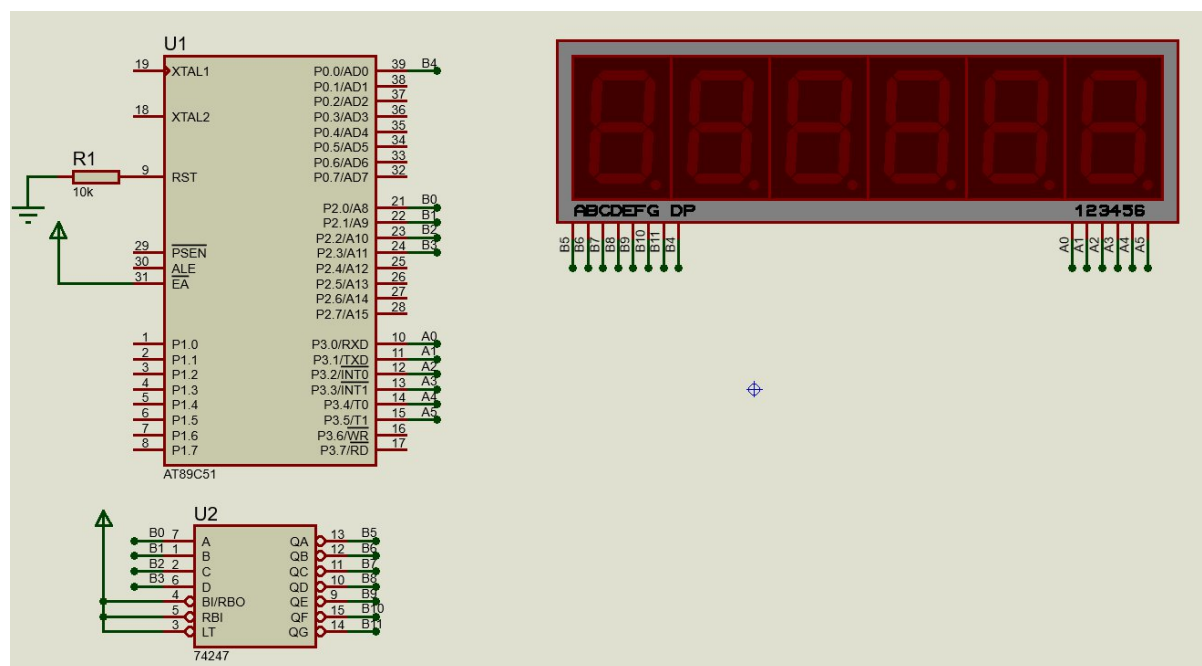
a. Flowchart of the LED scanning algorithm applied to display 7SEG-LED.



Vì 8051 không đủ chân để điều khiển riêng biệt từng thanh của 6 LED 7 đoạn (cần tới 42 chân), nên thuật toán quét LED này cho phép điều khiển 6 LED chỉ bằng 10 chân (4 chân dữ liệu và 6 chân điều khiển nguồn), giúp tối ưu hóa phần cứng.

II. Sử dụng Timer của vi điều khiển 8051, thiết kế mạch đồng hồ định dạng 24h với thời gian ban đầu được thiết lập sẵn trong mã nguồn.

a. Hình ảnh thiết kế.



b. Nguyên lý hoạt động mạch:

Linh kiện:

- AT89C51.
- 74247: Chuyển đổi BCD sang hiển thị LED 7 đoạn
 - Chân A-D: Nhận giá trị BCD, A là bit LSB, D là MSB
 - Chân BI/RBO: Xóa hiển thị. Nối với GND thực hiện tắt LED với mọi đầu vào, nối với VCC vô hiệu hóa chức năng.
 - Chân RBI: Lược hiển thị số 0 vô nghĩa. Nối với GND (vd: 500 à 500; 005 à 5), nối với VCC vô hiệu hóa chức năng.
 - Chân LT: Dùng để kiểm tra các LED có hoạt động đúng chức năng hay không. Nối GND thực hiện kiểm tra LED (Hiển thị số 8), nối với VCC vô hiệu hóa chức năng.
- 7SEG – MPX6 – CA: LED 7 đoạn 6 số
 - Chân A-G nối với các chân QA-QG của 74247. (Tích cực mức thấp)
 - Chân 1-6: Chọn LED muốn hiển thị. (Tích cực mức cao)
 - Chân DP: Hiển thị các dấu chấm.

c. Code sử dụng Timer

Assembly	Giải thích
<pre> ORG 0000H LJMP MAIN ORG 000BH LJMP IsrTimer0 ORG 0100H </pre>	Khởi tạo bộ nhớ cho chương trình. Gọi địa chỉ Vector ngắt cho Timer 0.
<pre> MAIN: MOV R0, #00 MOV R1, #56 MOV R2, #23 MOV R3, #0 ;Bien dem MOV R4, #0 MOV TMOD, #02H ;Timer0 mode 2 MOV TH0, #06H MOV TL0, #06H SETB EA SETB ET0 SETB TR0 SETB P1.0 </pre>	MOV R0, #00 (Tương tự với R1, R2): Thiết lập giá trị ban đầu cho Giây, Phút, Giờ. - R0 = 0 (Giây), R1 = 56 (Phút), R2 = 23 (Giờ). MOV TMOD, #02H: Cấu hình chế độ hoạt động cho bộ định thời. #02H tương đương với nhị phân 0000 0010. - Lệnh này chọn Timer 0, Chế độ 2 (8-bit tự động nạp lại giá trị). MOV TH0, #06H và MOV TL0, #06H: Thiết lập thời gian tràn. - Giá trị #06H làm cho Timer tràn sau mỗi 250 xung ($256 - 6 = 250$). - Với thạch anh 12MHz, lệnh này tạo ra một ngắt sau mỗi 250μs. SETB EA, SETB ET0, SETB TR0: Kích hoạt hệ thống ngắt. - EA: Cho phép ngắt toàn cục. - ET0: Cho phép ngắt Timer 0. - TR0: Bắt đầu cho phép Timer 0 đếm xung.

<pre> LOOP: LCALL DISPLAY SJMP LOOP DISPLAY: MOV A, R0 MOV B, #10 DIV AB MOV P2, B SETB P3.5 LCALL DELAY CLR P3.5 MOV A, R0 MOV B, #10 DIV AB MOV P2, A SETB P3.4 LCALL DELAY CLR P3.4 MOV A, R1 MOV B, #10 DIV AB MOV P2, B SETB P3.3 CLR P0.0 LCALL DELAY CLR P3.3 SETB P0.0 MOV A, R1 MOV B, #10 DIV AB MOV P2, A </pre>	LCALL DISPLAY & SJMP LOOP: Tạo vòng lặp hiển thị. • LCALL: Gọi chương trình con để xử lý dữ liệu. • SJMP: Nhảy ngược lại nhãn LOOP để lặp lại quá trình quét LED liên tục, giúp mắt người thấy số sáng cố định trên màn hình. MOV A, R0 (Tương tự với R1, R2): Di chuyển dữ liệu thời gian. • Lấy giá trị từ các thanh ghi lưu trữ (R0: Giây, R1: Phút, R2: Giờ) vào thanh ghi lũy thừa A để thực hiện phép toán. MOV B, #10 & DIV AB: Tách hàng đơn vị và hàng chục. • Chia giá trị thời gian cho 10. • Kết quả: Thương số (nằm ở A) là chữ số hàng chục, số dư (nằm ở B) là chữ số hàng đơn vị. MOV P2, B (hoặc MOV P2, A): Xuất dữ liệu ra Port 2. • Gửi giá trị số vừa tách được ra các chân P2.0 - P2.3. • IC giải mã 74247 nhận 4 bit này để điều khiển các thanh LED tương ứng. SETB P3.5 (Tương tự cho P3.0 đến P3.4): Kích hoạt chân điều khiển. • Đưa chân P3.x lên mức cao
--	--

<pre> SETB P3.2 LCALL DELAY CLR P3.2 MOV A, R2 MOV B, #10 DIV AB MOV P2, B SETB P3.1 CLR P0.0 LCALL DELAY CLR P3.1 SETB P0.0 MOV A, R2 MOV B, #10 DIV AB MOV P2, A SETB P3.0 LCALL DELAY CLR P3.0 </pre>	(5V) để "cấp nguồn" cho đúng 1 con LED cần hiển thị tại thời điểm đó. Đây là kỹ thuật quét LED (Multiplexing). CLR P0.0 & SETB P0.0: Điều khiển dấu chấm. - CLR P0.0: Đưa chân P0.0 về mức thấp (0V) để làm sáng dấu chấm ngăn cách giữa Giờ/Phút/Giây. - SETB P0.0: Tắt dấu chấm sau khi kết thúc chu kỳ hiển thị của LED đó. LCALL DELAY: Duy trì thời gian sáng. - Giữ cho LED hiện tại sáng trong một khoảng thời gian để người dùng có thể nhìn thấy. CLR P3.5 (Tương tự cho P3.0 - P3.4): Tắt LED hiện tại. - Đưa chân điều khiển về mức thấp (0V) để tắt LED trước khi chuyển sang LED tiếp theo, giúp hình ảnh sắc nét, không bị nhòe (hiện tượng bóng ma).
<pre> DELAY: MOV R7, #50 D1:DJNZ R7, D1 RET </pre>	MOV R7, #50: Thiết lập số vòng lặp đếm ngược. Sử dụng thanh ghi R7 làm bộ đếm. Giá trị #50 quyết định khoảng thời gian duy trì

<pre> IsrTimer0: INC R3 MOV A, R3 CJNE A, #40, EXIT_ISR MOV R3, #0 INC R4 MOV A, R4 CJNE A, #100, EXIT_ISR MOV R4, #0 INC R0 MOV A, R0 CJNE A, #60, EXIT_ISR MOV R0, #0 INC R1 MOV A, R1 CJNE A, #60, EXIT_ISR MOV R1, #0 INC R2 MOV A, R2 CJNE A, #24, EXIT_ISR MOV R2, #0 EXIT_ISR: RETI </pre>	trạng thái sáng của mỗi LED. D1: DJNZ R7, D1: Vòng lặp tạo độ trễ tại chỗ. DJNZ Mỗi vòng lặp sẽ giảm R7 đi 1 đơn vị. Nếu R7 vẫn khác 0, CPU sẽ nhảy ngược lại nhãn D1. RET: Lệnh quay về từ chương trình con. - Kết thúc hàm DELAY và trả quyền điều khiển về cho chương trình DISPLAY để tiếp tục quét các LED còn lại. INC R3 & CJNE A, #40, EXIT_ISR: Bộ chia tần số cấp 1 (Tạo mốc 10ms). - Vì Timer 0 được cấu hình tràn mỗi 250μs, ta cần đếm đủ 40 lần tràn để có: 10ms - Nếu chưa đủ 40 lần, CPU nhảy đến EXIT_ISR để thoát ngắt ngay lập tức. INC R4 & CJNE A, #100, EXIT_ISR: Bộ chia tần số cấp 2 (Tạo mốc 1 giây). - Sử dụng R4 để đếm 100 lần của mốc 10ms. - Khi đủ 100 lần, R4 được xóa về 0 và bắt đầu quy trình cập nhật thời gian chính. INC R0 & CJNE A, #60, EXIT_ISR: Cập nhật đơn vị Giây. - Tăng giá trị thanh ghi R0.
--	---

	Nếu đạt mốc 60, R0 reset về 0 và lệnh tiếp theo sẽ thực hiện tăng Phút. INC R1 & CJNE A, #60, EXIT_ISR: Cập nhật đơn vị Phút. - Tăng giá trị thanh ghi R1. Nếu đạt mốc 60, R1 reset về 0 và chuẩn bị tăng Giờ. INC R2 & CJNE A, #24, EXIT_ISR: Cập nhật đơn vị Giờ. - Tăng giá trị thanh ghi R2. Vì đồng hồ chạy định dạng 24h, nên khi đạt mức 24, giá trị sẽ quay về 0 để bắt đầu một ngày mới. EXIT_ISR: RETI: Lệnh quay về từ ngắt. RETI (Return from Interrupt): Kết thúc xử lý ngắt, khôi phục trạng thái và đưa CPU quay lại đúng vị trí đang quét LED trong chương trình chính.
--	--

III. Sử dụng Loop của vi điều khiển 8051, thiết kế mạch đồng hồ định dạng 24h với thời gian ban đầu được thiết lập sẵn trong mã nguồn.

Assembly	Giải thích
ORG 0000H LJMP MAIN ORG 0100H	Khởi tạo bộ nhớ cho chương trình.
MAIN: MOV R0, #00 MOV R1, #56 MOV R2, #23	MOV R0, #00 (Tương tự với R1, R2): Thiết lập giá trị ban đầu cho Giây, Phút, Giờ. R0 = 0 (Giây), R1 = 56

<pre> START_COUNT: MOV R5, #3 LOOP1: MOV R4, #128 LOOP2: LCALL DISPLAY DJNZ R4, LOOP1 DJNZ R5, LOOP2 </pre>	(Phút), R2 = 23 (Giờ). MOV R5, #3 & MOV R4, #128: Thiết lập bộ đếm lồng nhau. - Tạo ra tổng cộng 384 lần lặp. Dùng hai thanh ghi vì giới hạn của thanh ghi A là 255 (8-bit). LCALL DISPLAY: Hiển thị liên tục. - Gọi chương trình quét LED ngay trong lúc chờ. Điều này giữ cho đèn LED luôn sáng và không bị tắt trong khoảng thời gian 1 giây đó. DJNZ R4, LOOP1: Vòng lặp con (bên trong). - Giảm R4 và lặp lại việc quét LED 100 lần. Đây là nòng cốt của việc tạo trễ nhỏ. DJNZ R5, LOOP2: Vòng lặp chính (bên ngoài). - Sau khi R4 về 0, R5 giảm 1 và nạp lại R4 để chạy tiếp. Khi cả hai về 0, vi điều khiển xác nhận đã trôi qua ~1 giây. START_COUNT: Bắt đầu chu kỳ mới. - Nhấn đánh dấu điểm reset bộ đếm trễ để chuẩn bị cho giây tiếp theo sau khi đã tăng các giá trị thời gian.
DISPLAY:	LCALL DISPLAY & SJMP

MOV A, R0 MOV B, #10 DIV AB MOV P2, B SETB P3.5 LCALL DELAY CLR P3.5 MOV A, R0 MOV B, #10 DIV AB MOV P2, A SETB P3.4 LCALL DELAY CLR P3.4 MOV A, R1 MOV B, #10 DIV AB MOV P2, B SETB P3.3 CLR P0.0 LCALL DELAY CLR P3.3 SETB P0.0 MOV A, R1 MOV B, #10 DIV AB MOV P2, A SETB P3.2 LCALL DELAY CLR P3.2 MOV A, R2	LOOP: Tạo vòng lặp hiển thị. • LCALL: Gọi chương trình con để xử lý dữ liệu. • SJMP: Nhảy ngược lại nhãn LOOP để lặp lại quá trình quét LED liên tục, giúp mắt người thấy số sáng cố định trên màn hình. MOV A, R0 (Tương tự với R1, R2): Di chuyển dữ liệu thời gian. • Lấy giá trị từ các thanh ghi lưu trữ (R0: Giây, R1: Phút, R2: Giờ) vào thanh ghi lũy thừa A để thực hiện phép toán. MOV B, #10 & DIV AB: Tách hàng đơn vị và hàng chục. • Chia giá trị thời gian cho 10. • Kết quả: Thương số (nằm ở A) là chữ số hàng chục, số dư (nằm ở B) là chữ số hàng đơn vị. MOV P2, B (hoặc MOV P2, A): Xuất dữ liệu ra Port 2. • Gửi giá trị số vừa tách được ra các chân P2.0 - P2.3. • IC giải mã 74247 nhận 4 bit này để điều khiển các thanh LED tương ứng. SETB P3.5 (Tương tự cho P3.0 đến P3.4): Kích hoạt chân điều khiển. • Đưa chân P3.x lên mức cao (5V) để "cấp nguồn" cho đúng 1 con LED cần hiển thị
---	--

<pre> MOV B, #10 DIV AB MOV P2, B SETB P3.1 CLR P0.0 LCALL DELAY CLR P3.1 SETB P0.0 MOV A, R2 MOV B, #10 DIV AB MOV P2, A SETB P3.0 LCALL DELAY CLR P3.0 </pre>	tại thời điểm đó. Đây là kỹ thuật quét LED (Multiplexing). CLR P0.0 & SETB P0.0: Điều khiển dấu chấm. - CLR P0.0: Đưa chân P0.0 về mức thấp (0V) để làm sáng dấu chấm ngăn cách giữa Giờ/Phút/Giây. - SETB P0.0: Tắt dấu chấm sau khi kết thúc chu kỳ hiển thị của LED đó. LCALL DELAY: Duy trì thời gian sáng. - Giữ cho LED hiện tại sáng trong một khoảng thời gian để người dùng có thể nhìn thấy. CLR P3.5 (Tương tự cho P3.0 - P3.4): Tắt LED hiện tại. - Đưa chân điều khiển về mức thấp (0V) để tắt LED trước khi chuyển sang LED tiếp theo, giúp hình ảnh sắc nét, không bị nhòe (hiện tượng bóng ma).
<pre> DELAY: MOV R7, #50 D1:DJNZ R7, D1 RET INC R0 MOV A, R0 </pre>	MOV R7, #50: Thiết lập số vòng lặp đếm ngược. Sử dụng thanh ghi R7 làm bộ đếm. Giá trị #50 quyết định khoảng thời gian duy trì trạng thái sáng của mỗi LED. D1: DJNZ R7, D1: Vòng lặp

<pre> CJNE A, #60, EXIT_ISR MOV R0, #0 INC R1 MOV A, R1 CJNE A, #60, EXIT_ISR MOV R1, #0 INC R2 MOV A, R2 CJNE A, #24, EXIT_ISR MOV R2, #0 END </pre>	tạo độ trễ tại chỗ. • DJNZ Mỗi vòng lặp sẽ giảm R7 đi 1 đơn vị. Nếu R7 vẫn khác 0, CPU sẽ nhảy ngược lại nhãn D1. INC R0 & CJNE A, #60, EXIT_ISR: Cập nhật đơn vị Giây. • Tăng giá trị thanh ghi R0. Nếu đạt mốc 60, R0 reset về 0 và lệnh tiếp theo sẽ thực hiện tăng Phút. INC R1 & CJNE A, #60, EXIT_ISR: Cập nhật đơn vị Phút. • Tăng giá trị thanh ghi R1. Nếu đạt mốc 60, R1 reset về 0 và chuẩn bị tăng Giờ. INC R2 & CJNE A, #24, EXIT_ISR: Cập nhật đơn vị Giờ. • Tăng giá trị thanh ghi R2. Vì đồng hồ chạy định dạng 24h, nên khi đạt mức 24, giá trị sẽ quay về 0 để bắt đầu một ngày mới. EXIT_ISR: RETI: Lệnh quay về từ ngắt. • RETI (Return from Interrupt): Kết thúc xử lý ngắt, khôi phục trạng thái và đưa CPU quay lại đúng vị trí đang quét LED trong chương trình chính.
--	--

Đánh giá và so sánh hai phương pháp:

- **Phương pháp Timer:**

- **Ưu điểm:**

- **Độ chính xác cao:** Nhờ thạch anh và timer cứng của MCU.
- **Tính chất "Non-blocking":** CPU rảnh rỗi để thực hiện các tác vụ khác như quét hiển thị 6 LED liên tục, giúp hình ảnh mượt mà, không chớp nháy mà vẫn đảm bảo thời gian chuẩn xác.
- **Đa nhiệm hiệu quả:** Cho phép hệ thống xử lý nhiều công việc cùng lúc (vừa đếm giờ, vừa quét phím, vừa giao tiếp dữ liệu).
- **Dễ mở rộng code:** Dễ kết hợp thêm tính năng khác song song.
- **Tính ổn định lâu dài:** Không bị trôi nhịp, không lệ thuộc vào độ dài code trong main loop.

- **Nhược điểm:**

- **Rủi ro xung đột dữ liệu:** Do luồng chương trình chạy bất đồng bộ, ngắt có thể xảy ra bất cứ lúc nào, dễ gây ra lỗi ghi đè dữ liệu nếu người lập trình không quản lý tốt việc cất/lấy dữ liệu (Stack).
- **Phức tạp cao:** Đòi hỏi người lập trình phải hiểu sâu về cấu trúc ngắt, cách tính toán giá trị nạp (TH, TL) và cấu hình các thanh ghi liên quan.

• **Phương pháp sử dụng Loop:**

- **Ưu điểm:**

- **Cấu trúc đơn giản:** Mã nguồn dễ viết, dễ hiểu, phù hợp cho các ứng dụng cực kỳ đơn giản hoặc người mới bắt đầu.
- **Tính tuần tự:** Chương trình chạy từ trên xuống dưới, không xảy ra hiện tượng xung đột tài nguyên (không bị ngắt bất ngờ làm sai lệch dữ liệu trong thanh ghi A hoặc các thanh ghi tạm khi đang tính toán hiển thị).
- **Không phụ thuộc phần cứng:** Không cần cấu hình các thanh ghi chức năng đặc biệt (SFR) của Timer.

- **Nhược điểm:**

- **Tính chất "Blocking":** CPU bị chiếm dụng hoàn toàn trong suốt thời gian trễ, không thể thực hiện bất kỳ tác vụ nào khác (như kiểm tra nút bấm hay nhận tín hiệu ngoại vi).
- **Sai số lớn:** Độ chính xác phụ thuộc hoàn toàn vào số lượng lệnh và tần số thạch anh. Chỉ cần thay đổi một dòng code trong hàm hiển thị, thời gian trễ sẽ bị lệch, dẫn đến đồng hồ chạy sai lệch đáng kể qua thời gian.
- **Khó mở rộng code nếu cần.**

Video mô phỏng hoạt động của mạch:

https://drive.google.com/drive/folders/1C3zoXUcoQFcT5Td-ucQtb_PG5FjfR6oi?usp=sharing